

# Nesne Tabanlı Programlamada Proje Yönetimi

NTP II - Hafta 7



# Nesne Tabanlı Programlamada Proje Yönetimi

Bu sunumda, modern yazılım geliştirmede karmaşıklığı yönetmek için hayati öneme sahip iki temel konsepti inceleyeceğiz: Modüller ve Sanal Ortamlar (venv).

Gündemimiz:

1. Kodun Yapılandırılması: Modülerliğin Temelleri
2. Bağımlılık Yönetimi ve 'Kütüphane Cehennemi' Sorunu
3. PIP: Python Paket Yöneticisi
4. Sanal Ortamlar (venv): İzolasyonun Önemi



Nesne Tabanlı Programlama (OOP), günümüzde kullanılan çoğu sistemin temelini oluşturur.

OOP, karmaşık sistemleri yönetilebilir, bakımı kolay ve yeniden kullanılabilir kod parçalarına ayırmayı sağlar.

Bu ders, OOP ilkelerini büyük ölçekli ve güvenli yazılım projelerine nasıl uygulayacağımıza odaklanmaktadır.



# Yazılım Karmaşıklığının Yönetimi

Yazılım projeleri büyüdükçe, tek bir dosya veya monolitik bir yapı sürdürülemez hale gelir.  
Büyük kod bloklarında hata ayıklama zorlaşır ve yeni özellik ekleme riski artar.  
Çözüm: Kodları farklı dosyalara bölme (modüller) yaklaşımıdır.

# Modülerlik Nedir?

Modül, ilgili işlevleri, sınıfları ve değişkenleri bir araya getiren tek bir dosyadır. Python'da her '.py' dosyası bir modül olarak işlev görebilir. Modüller, kodun mantıksal olarak gruplandırılmasını sağlar, bu da okunabilirliği ve test edilebilirliği artırır.



# OOP ve Modüllerin Entegrasyonu

Nesne Tabanlı Programlamada, farklı sorumlulukları olan sınıfları ayrı modüllere yerleştirmek yaygın bir en iyi uygulamadır. Örnek olarak, bir insan kaynakları sisteminde, kullanıcılarla ilgili tüm sınıflar (Insan, Ogrenci, Ogretmen) tek bir 'modeller.py' dosyasına taşınabilir.



# Uygulama: models.py Dosyasının Oluşturulması

Proje Yönetimi kuralına göre, tüm sınıfları (Insan, Ogrenci, Ogretmen) ayrı bir 'modeller.py' dosyasına taşıyın.

Bu, ana program mantığı ile veri yapılarını birbirinden ayırır.

modeller.py dosyası sadece bu sınıfların tanımlarını içermelidir.

Bu sayede, ana program (main.py) çok daha temiz ve odaklanmış kalır.



# İçe Aktarma (Import) Mekanizması

Bir modülde tanımlanan sınıfları veya işlevleri kullanabilmek için, ana dosyanızdan import etmeniz gerekir.

Temel import formları:

1. import modeller (Tüm modülü içe aktarma)
2. from modeller import Öğrenci (Belirli bir sınıfı içe aktarma)
3. import modeller as m (Takma adla içe aktarma)

# Namespace (İsim Alanı) Yönetimi

Modüller, kendi isim alanlarını (namespace) oluşturarak, farklı modüllerde aynı isme sahip değişkenlerin veya fonksiyonların çatışmasını önler. Örneğin, hem 'modul\_a' hem de 'modul\_b' içinde 'hesapla' adında bir fonksiyon olabilir; import ederken bu ayırım korunur.

# Paketler ve Hiyerarşik Yapı

Paket, ilgili modülleri bir araya getiren bir dizindir (klasör). Örneğin, bir 'utils' paketi birçok yardımcı modül içerebilir.

Paketler, büyük projelerde organizasyonu sağlar ve import yollarını basitleştirir. Python 3.3'ten bu yana, bir dizinin paket olarak kabul edilmesi için mutlaka '\_\_init\_\_.py' dosyasına sahip olması gerekmez, ancak bu dosya hala paket başlatma kodunu tutmak için kullanılır.

# Yazılım Tasarımında Modülerlik İlkeleri

**Yüksek Kayıt (High Cohesion):** Bir modül içindeki elementler birbiriyle sıkıca ilişkilidir ve tek bir göreve odaklanmıştır.

**Düşük Bağlılık (Low Coupling):** Modüller arasındaki bağımlılık mümkün olduğunca azdır.

Bu iki ilke, modülerliği başarılı kılan temel akademik yaklaşımlardır.



# Modülerliğin Avantajları (1/2)

1. Bakım Kolaylığı: Bir hata bulunduğunda, yalnızca ilgili modül üzerinde çalışılması gerekir.
2. Yeniden Kullanılabilirlik: İyi tasarlanmış bir modül, farklı projelerde veya projenin farklı kısımlarında tekrar kullanılabilir.
3. Okunabilirlik: Daha küçük, odaklanmış dosyalar sayesinde kodun anlaşılması kolaylaşır.

## Modülerliğin Avantajları (2/2)

4. Test Edilebilirlik: Modüller izole olduğu için, her biri bağımsız birim testlerine tabi tutulabilir.
5. Eş Zamanlı Geliştirme: Farklı geliştiriciler, birbirlerinin kodlarını engellemeden aynı anda farklı modüller üzerinde çalışabilirler.

# Proje Bağımlılıkları Kavramı

Modern yazılım geliştirme, sıfırdan yazılmayan, dışarıdan alınan kütüphanelere dayanır.

Bu dış kütüphanelere 'bağımlılıklar' denir.

Örneğin, web istekleri yapmak için 'requests' kütüphanesini kullanma ihtiyacı duyulur.



# Bağımlılık Yönetiminin Kritik Önemi

Bir kütüphanenin farklı sürümleri, farklı fonksiyon isimlerine, davranışlara veya bağımlılıklara sahip olabilir.  
Eğer bir proje, bir kütüphanenin belirli bir sürümüne bağlıysa, bu sürümün korunması gerekir.



# 'Kütüphane Cehennemi' (Dependency Hell) Sorunu

Kütüphane cehennemi, iki farklı uygulamanın (Proje A ve Proje B) aynı kütüphanenin farklı ve uyumsuz sürümlerine ihtiyaç duymasıyla ortaya çıkar. Eğer kütüphaneler sistemin ana (küresel) ortamına kurulursa, bir projenin gereksinimleri diğerini bozar. Bu, özellikle büyük sunucu ortamlarında felaketlere yol açabilir.

# PIP: Python Paket Yöneticisi Tanımı

PIP, Python için standart paket yöneticisidir.  
PyPI'da (Python Paket Dizini) barındırılan binlerce kütüphaneyi komut satırı üzerinden kurmayı, yükseltmeyi ve kaldırmayı sağlar.  
PIP, bir geliştiricinin herhangi bir Python projesinin temel araçlarından biridir.



# PyPI (Python Package Index)

PyPI, üçüncü taraf Python yazılım paketlerinin resmi deposudur. PIP, kurulum işlemlerini gerçekleştirmek için varsayılan olarak PyPI ile iletişim kurar. Bu sayede, 'requests' gibi kütüphaneleri kolayca bulabilir ve projenize dahil edebilirsiniz.



# PIP Temel Komutları

Kurulum: `pip install [paket_adı]` (Örnek: `pip install requests`)

Kaldırma: `pip uninstall [paket_adı]`

Listeleme: `pip list` (Kurulu tüm paketleri gösterir)

Sürüm Bilgisi: `pip show [paket_adı]`



# Örnek Uygulama: 'requests' Kütüphanesi

requests kütüphanesi, insan dostu HTTP istekleri yapmak için popüler bir kütüphanedir.

Bu kütüphaneyi kurun emri, herhangi bir modern Python projesinde bağımlılık yönetiminin ne kadar kritik olduğunu gösterir.

Kurulumu, pip install requests komutuyla yapılır.



# PIP ile Sürüm Belirleme (Pinning)

İstikrarı sağlamak için, projenizin ihtiyaç duyduğu kütüphanelerin tam sürümlerini belirtmek önemlidir.

Örnek: `pip install requests==2.28.1`

Bu, gelecekteki olası uyumsuzlukları önler ve projenizin farklı makinelerde aynı şekilde çalışmasını garanti eder.

# requirements.txt Dosyasının Kullanımı

requirements.txt, bir projenin tüm dış bağımlılıklarını ve gerekli sürümlerini listeleyen standart metin dosyasıdır.

Bu dosya, projenin taşınabilirliğini sağlar.

Komut: `pip install -r requirements.txt` (listedeki tüm paketleri kurar).

# PIP Freeze Komutu

pip freeze komutu, aktif sanal ortamda (veya küresel ortamda) kurulu olan tüm paketleri ve tam sürüm numaralarını requirements.txt formatında çıktı verir.

Bu çıktı, projenin bağımlılıklarını kaydetmek için kullanılır.

Komut: `pip freeze > requirements.txt`

# Sanal Ortamlar (Virtual Environments) Nedir?

Sanal ortamlar (venv), bir Python projesinin kendi bağımlılıklarını diğer projelerin bağımlılıklarından tamamen izole etmesini sağlayan hafif, kendi kendini kapsayan izin ağacıdır.

Temel amaç, 'kütüphane cehennemi' sorununu kalıcı olarak çözmektir.



# Neden Sanal Ortamlara İhtiyaç Duyulur?

Sistem Düzeyi Kütüphanelerinden Kaçınma: Sisteminizin işletim sistemi veya temel araçları için kritik olan kütüphanelerin yanlışlıkla bozulmasını önler.

Bağımsızlık: Her proje, ihtiyaç duyduğu spesifik Python yorumlayıcısı ve kütüphane sürümüne sahip olabilir.

# venv Modülü ve Oluşturma Süreci

Python 3.3+'dan beri venv, Python Standart Kütüphanesinin bir parçasıdır. Ortam oluşturma komutu: `python -m venv [ortam_adı]`  
Bu komut, belirtilen dizinde (örneğin `.venv` veya `env`) gerekli dosyaları ve Python yorumlayıcısının bir kopyasını oluşturur.

# Sanal Ortamın Temel Bileşenleri

Bir sanal ortam dizini şunları içerir:

1. bin (veya Scripts): Ortamın Python yorumlayıcısı ve pip gibi yardımcı programları.
  2. lib (veya Lib): Bu ortama özel olarak kurulan tüm kütüphaneler (örneğin requests kütüphanesi buraya kurulur).
- Global ortamdan tamamen ayrılmış bir kopya yaratılır.

# Ortaman Aktifleştirilmesi (Activation)

Sanal ortamı kullanmaya başlamak için aktifleştirilmesi gerekir. Aktifleştirme, kabuk (shell) değişkenlerini (özellikle PATH) değiştirerek, 'python' ve 'pip' komutlarının artık global olanlar yerine ortamın kendi kopyasını işaret etmesini sağlar.



# Aktif Ortamda PIP Kullanımı

Sanal ortam aktifleştirildikten sonra, pip install requests komutu çalıştırıldığında, bu kütüphane sadece aktif ortama kurulur. Böylece, sistemdeki diğer projeler veya global kütüphaneler etkilenmez. Her proje, kendi bağımlılıklarını yönetmesini sağlar.

# Ortamin Deaktiflestirilmesi

Proje üzerindeki çalışmanız bittiğinde, deaktifleştirme komutuyla sanal ortamdan çıkılır.

Komut: deactivate

Bu, PATH değişkenini sıfırlayarak global Python kurulumuna geri dönmenizi sağlar.



# Sanal Ortamların 'Kütüphane Cehennemi' Çözümü

Proje A, Kütüphane X sürüm 1.0'ı gerektirebilir.

Proje B, Kütüphane X sürüm 2.0'ı gerektirebilir.

Sanal Ortamlar, bu iki projenin bağımsız çalışmasına izin vererek çatışmayı önler.

Her projenin kendi bağımlılıklarını yönetmesi temel çözüm mekanizmasıdır.

# Sanal Ortamların Proje Taşınabilirliği Üzerindeki Etkisi

Sanal ortamlar ve requirements.txt birlikte kullanıldığında, bir projenin tam olarak aynı bağımlılıklarla herhangi bir makinede yeniden oluşturulması garanti edilir.

Bu, DevOps süreçlerinde ve sürekli entegrasyon (CI) boru hatlarında kritik öneme sahiptir.



# Gelişmiş Sanal Ortam Yöneticileri

venv basit ve standart bir çözüm olsa da, daha karmaşık projeler için farklı araçlar mevcuttur:

1. Conda: Genellikle veri bilimi projelerinde kullanılan, sadece Python değil, sistem kütüphanelerini de yönetebilen bir paket yöneticisidir.
2. Pipenv: Bağımlılıkları ve sanal ortamları tek bir araçta birleştiren popüler bir araçtır.

# Poetry: Modern Bağımlılık Yönetimi

Poetry, bağımlılık yönetimini ve Python paketlemeyi basitleştirmek için tasarlanmış nispeten yeni bir araçtır. requirements.txt yerine TOML formatlı 'pyproject.toml' dosyasını kullanır. Poetry, bağımlılıkları çözmede daha sofistike bir yaklaşım sunar ve daha güvenilir kilit dosyaları oluşturur.



# Sanal Ortamlar ve Güvenlik

Sanal ortamlar, bağımlılıkları izole ederek bir kütüphanenin potansiyel güvenlik açığının sistemin geri kalanına yayılmasını zorlaştırır. Eğer requests kütüphanesinde bir güvenlik açığı tespit edilirse, bu açık sadece ilgili sanal ortamda kurulu olan projeyi etkileyecektir, tüm sistemi değil.



# Sanal Ortamların Sürüm Kontrolü (Git) ile Yönetimi

Sanal ortam dizinleri (örneğin '.venv'), genellikle büyük oldukları ve her zaman yeniden oluşturulabildikleri için Sürüm Kontrol Sistemlerine (Git) dahil EDİLMEMELİDİR.

En iyi uygulama, bu dizinleri '.gitignore' dosyasına eklemektir. Sadece requirements.txt (veya pyproject.toml) dosyası paylaşılmalıdır.



# requirements.txt Oluşturma Uygulaması

Senaryo: Projemiz için requests kütüphanesini kurduk.

1. Ortamı aktifleştirin: `source env/bin/activate`

2. Kurulu kütüphaneleri dondurun: `pip freeze > requirements.txt`

Bu adımlar, projenin kesin bağımlılık setini güvence altına alır.



# İçe Aktarma Yollarının Derinlemesine İncelenmesi

Bir import işlemi gerçekleştiğinde, Python sys.path listesindeki dizinleri sırayla kontrol eder.

Sanal ortam aktifleştirildiğinde, ortamın kütüphane dizini (lib/site-packages) sys.path'in en üstüne eklenir.

Bu, yerel kütüphanelerin global kütüphanelerden önce bulunmasını sağlar.



# Geliştirme (Dev) ve Üretim (Prod) Bağımlılıkları

Bazı kütüphaneler (örneğin test çerçeveleri, kod formatlayıcılar) sadece geliştirme sırasında gereklidir. Üretim ortamının gereksiz bağımlılıklarla şişmesini önlemek için bu listeler ayrılmalıdır. Gelişmiş araçlar (Poetry, Pipenv) bu ayrımı otomatik olarak yönetebilir.



# Farklı Python Sürümlerini Yönetmek

Bir proje Python 3.8, diğeri Python 3.11 gerektirebilir.

Sanal ortamlar, hangi Python yorumlayıcısının kullanılacağını seçerek oluşturulur.

Komut: `/usr/bin/python3.8 -m venv env_proje_a`

Bu özellik, eski kod tabanlarını korurken yeni teknolojileri denemeye olanak tanır.



# Sanal Ortamların Bir Adım Ötesi: Konteynerler

Sanal ortamlar sadece Python bağımlılıklarını izole ederken, Docker gibi konteyner teknolojileri tüm işletim sistemi seviyesini (veritabanları, web sunucuları dahil) izole eder.

Konteynerler, sanal ortamları tamamlayıcı olarak kullanılır, rekabet edici değildir.

Konteynerler, en üst düzeyde ortam tutarlılığı sağlar.



# Nesne Tabanlı Modüllerin Tasarımı

OOP sınıflarını ayırırken (Insan, Ogrenci, Ogretmen), her bir modülün tek bir sorumluluğu olduğundan emin olun (Single Responsibility Principle).  
modeller.py sadece veri modellerini tutmalıdır; veritabanı bağlantılarını başka bir 'db\_handler.py' modülüne bırakmalıdır.

# Modüler Programlamada Hata Ayıklama

Modüler bir yapı, bir hata oluştuğunda sorunun hangi modülden kaynaklandığını hızla tespit etmeyi sağlar.  
Örneğin, sadece 'modeller.py' dosyasının testleri başarısız olursa, hatanın kapsamı daralır.

# PIP Güvenlik Riskleri

PyPI'a herkes paket yükleyebilir, bu nedenle rastgele veya yazım hatası içeren paketleri kurmak (typosquatting) güvenlik riskleri taşır.

Hata: pip install requesst (sadece bir 's' eksik). Bu, saldırganın paketini yükleyebilir.

Her zaman güvenilir ve geniş çapta kullanılan kütüphaneleri kurun.



# Örnek Senaryo: Veri Çekme Projesi

Bir web sitesinden veri çekmek için bir proje başlatıyorsunuz.

1. Sanal ortam oluşturun ve aktifleştirin.
  2. 'pip install requests' komutuyla gerekli kütüphaneyi kurun.
  3. Veri işleme mantığını ayrı bir 'scraper.py' modülüne taşıyın.
- Bu yapı, projenin temiz ve izole kalmasını sağlar.

# Kapsamlı Proje Yönetimi Kontrol Listesi

1. Sanal Ortam Oluşturun (venv).
2. Sanal Ortamı Aktifleştirin.
3. Gerekli Dış Kütüphaneleri PIP ile kurun (örneğin requests).
4. Kodunuzu Sınıf ve İşlev Sorumluluklarına göre Modüllere Ayırın (örneğin modeller.py).
5. pip freeze > requirements.txt ile bağımlılıkları kaydedin.
6. Ortam dizinini (.gitignore) ile hariç tutun.

# Nesne Tabanlı Programlamada Modüllerin Rolü

Modüller, sadece dosyaları bölmekten öte, OOP'nin soyutlama (abstraction) ve kapsülleme (encapsulation) ilkelerini fiziksel olarak uygulama yöntemidir. Her modül, belirli bir soyutlamayı temsil eder ve içerdği sınıflar o soyutlamanın detaylarını kapsüller.



# Dinamik İe Aktarma ve Performans

Normalde tm modller program bařladıėında yklenir. Ancak, bazı durumlarda (rneėin hız optimizasyonu veya dairesel baėımlılıkları kırma), modllerin programın alıřması sırasında dinamik olarak yklenmesi gerekebilir. Bu, genellikle yksek performanslı uygulamalarda dikkate alınması gereken ileri dzey bir konudur.



# PIP Çözümleyicisinin Çalışma Prensibi

Bir kütüphane kurduğunuzda, PIP bu kütüphanenin de bağlı olduğu alt bağımlılıkları (transitive dependencies) belirler.  
PIP, bu bağımlılıkların tümü arasında sürüm çatışması olup olmadığını kontrol eden bir çözümleyici (resolver) kullanır.  
Başarısız bir çözümleme, uyumsuz sürüm gereksinimlerinden kaynaklanır.

# Sanal Ortamlarla Kullanıcı İzinleri

Sanal ortamlar, bağımlılıkları kurmak için sistem düzeyinde 'sudo' veya yönetici izinleri gereksinimini ortadan kaldırır.  
Kullanıcılar, kendi ortamlarına istedikleri kütüphaneyi kurabilirler.  
Bu, sistem güvenliğini artırır çünkü geliştiricilerin ana sisteme müdahale etmesi gerekmez.



# Üniversite Projelerinde Sanal Ortam Zorunluluğu

Üniversite düzeyinde projelerin tesliminde sanal ortamların kullanılması bir standart olmalıdır.

Bu, proje değerlendirme ortamının, öğrencinin yerel makinesindeki ortamla tutarlı olmasını sağlar.

requirements.txt dosyası, projenin 'DNA'sıdır.



# Modüllerin Kalite Kontrolü

Flake8 veya Pylint gibi araçlar, modül içindeki kod kalitesini ve stil tutarlılığını kontrol etmek için kullanılır.

Bu araçlar, sanal ortama kurularak sadece o projeye özel kurallarla çalıştırılabilirler.



# Sanal Ortamların Bellek ve Depolama Etkisi

Her sanal ortam, Python yorumlayıcısının ve standart kütüphanelerin kopyasını içerir, bu da diskte yer kaplar.  
Ancak modern disk kapasiteleri göz önüne alındığında, sağladığı izolasyon faydası bu maliyeti fazlasıyla karşılar.  
Sadece aktif olarak kullanılan ortamlar sistem belleğinde yer tutar.



# Ortam Değişkenlerinin Rolü

Hassas bilgiler (API anahtarları, veritabanı şifreleri) genellikle sanal ortam içindeki bir '.env' dosyası veya kabuk ortam değişkenleri aracılığıyla yönetilir. Bu, kodun kendisinde hassas verilerin bulunmasını engelleyen bir güvenlik uygulamasıdır.



# İmport Çözümlemede Mutlak ve Görelî Yollar

Mutlak İçe Aktarma: Kök paketten başlar (örneğin: `from models import Ogrenci`). Bu daha güvenlidir.

Görelî İçe Aktarma: Mevcut modüle göre yol belirtir (örneğin: `from .utils import helper`). Genellikle paket içinde kullanılır.

Kaynakta belirtilen '`modeller.py`' içe aktarımı mutlak bir import örneğidir.

# Sanal Ortamların Otomatik Yönetimi (IDE'ler)

PyCharm, VS Code gibi modern Entegre Geliştirme Ortamları (IDE'ler), bir proje açıldığında sanal ortam oluşturma, aktifleştirme ve kütüphane kurma işlemlerini otomatik olarak yönetebilir.

Bu, geliştirme sürecini önemli ölçüde hızlandırır ve hataları azaltır.



# Projelerin Sürdürülmesi ve Güncelleme

Eski bir projeye geri dönüldüğünde, requirements.txt sayesinde sanal ortamı kolayca yeniden oluşturabiliriz.  
Ancak, kütüphanelerin büyük sürüm güncellemeleri (major version updates), kodda kırılmalara yol açabilir.  
Bu durumda, eski sanal ortamı kullanmaya devam etmek veya dikkatli bir geçiş planı yapmak gerekir.



# PIP Cache Mekanizması

PIP, indirilen paket dosyalarını yerel bir önbellekte (cache) tutar. Bir paketi birden fazla sanal ortama kurduğunuzda, PIP genellikle paketi yeniden indirmek yerine önbellekten kopyalar. Bu, hem ağ trafiğini azaltır hem de kurulum süresini kısaltır.



# Sanal Ortam ve Docker Karşılaştırması (Derin Bakış)

Sanal Ortam (venv): Uygulama katmanı izolasyonu. Ana işletim sistemi çekirdeğini kullanır.

Docker: İşletim sistemi seviyesi izolasyonu. Uygulama ve tüm bağımlılıkları (Python, işletim sistemi dosyaları) konteyner içinde paketler.

Konteynerler, daha tutarlı ve dağıtılabilir çözümler sunar, ancak daha fazla kaynak gerektirir.



# Bağımlılık Kilit Dosyaları (Lock Files)

Poetry ve Pipenv gibi araçlar, requirements.txt yerine kilit dosyaları (lock files) kullanır.

Kilit dosyaları, üst düzey bağımlılıkların yanı sıra tüm alt bağımlılıkların (transitive dependencies) tam sürüm ve hash değerlerini içerir. Bu, kurulumun her zaman aynı sonucu vermesini (deterministik) sağlar.



# Modül Giriş Noktası (`__main__.py`)

Bir paketin bir komut dosyası gibi çalıştırılabilmesi için, genellikle kök dizininde '`__main__.py`' dosyası kullanılır.  
'python -m [paket\_adı]' komutu, Python'a paketi modül olarak çalıştırmasını ve '`__main__.py`' dosyasını giriş noktası olarak kullanmasını söyler.

# Sanal Ortam Temizliği ve Yönetimi

Sanal ortamlar sadece bir dizindir; kullanılmadıklarında veya bozulduklarında basitçe silinebilirler.

Silme işlemi: `rm -rf env_adi` (Linux/macOS) veya dizini manuel silme (Windows).

Bu, sisteminizi temiz tutmanın ve depolama alanını optimize etmenin basit bir yoludur.



# Özet: Temel Kavramların Birleşimi

Bu sunumda öğrendiğimiz gibi, Nesne Tabanlı Programlama (OOP) ve Modüller, kodun iç organizasyonunu sağlar (modeller.py). PIP ve Sanal Ortamlar (venv) ise, kodun dış bağımlılıklarını izole ederek proje bütünlüğünü ve taşınabilirliğini garanti eder (requests kütüphanesi ve 'library hell' çözümü). Bu iki yaklaşım, modern yazılım mühendisliğinde ayrılmaz bir bütündür.

# Kilit Çıkarımlar

1. Modülerlik: Büyük projeler için hayati öneme sahiptir; sınıflar dosyalara ayrılmalıdır.
2. İzolasyon: Sanal ortamlar, 'kütüphane cehennemi' sorununa karşı tek çözümdür.
3. Standardizasyon: PIP ve requirements.txt kullanımı zorunludur.
4. Güvenlik: Bağımlılıklarınızı her zaman izole edin ve kontrol edin.

# Gelecek Adımlar ve İleri Konular

1. Paket Geliştirme ve PyPI'ya Yayınlama.
2. Docker kullanarak Python uygulamalarını konteynerize etme.
3. Sürekli Entegrasyon (CI) sistemlerinde sanal ortamların kullanımı.
4. Farklı geliştirme ortamları (dev, staging, prod) için farklı requirements dosyaları oluşturma.



# Teşekkürler

Konuyla ilgili sorularınızı veya proje uygulamalarınızdaki karşılaştığınız zorlukları tartışmaya açabiliriz.  
Hazırladığınız projelerde modül ve sanal ortamları doğru kullandığınızdan emin olun.



# Nesne Tabanlı Programlamada Proje Yönetimi

Süleyman **EZDEMİR**

*suleyman.ezdemir@giresun.edu.tr*

